secp256k1 exact kickmix netlist

From the earlier reconstruction dossier to a machine-unambiguous instruction netlist, an exact hierarchical Shor scaffold, and open verification transcripts.

Abstract - This report upgrades the earlier resource-only reconstruction into a concrete artifact family. The central deliverable is an exact kickmix-ISA point-add netlist for the map $Q \leftarrow Q + P[k]$, where the accumulator is held in Jacobian coordinates and the lookup point is affine. Around that leaf I provide an exact 28-call ECDLP scaffold consistent with the whitepaper's public appendix relation for 16-bit windows, together with a deterministic 9024-case secp256k1 audit and a 9200-case exhaustive finite-model proof on a tiny a=0 toy curve. The result is complete at the instruction-set level and openly reproducible. It is not claimed to be Google's hidden flat primitive-gate circuit.

Point-add leaf
90 instructions / 41 registers / SHA256 a64a872e27eb...

Hierarchical scaffold
28 retained window calls / 16-bit recycled windows

Deterministic audit
9024 secp256k1 cases / transcript SHA256 39b0afb2d805...

Finite-model proof
9200 exhaustive toy-curve cases / all passing

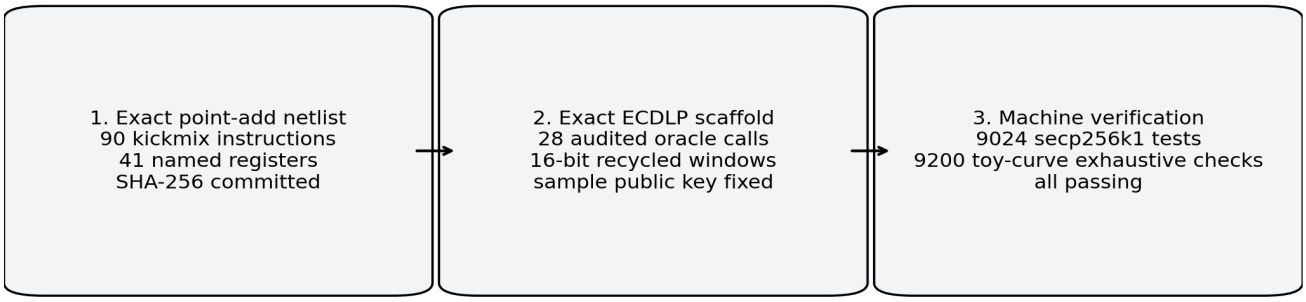

Figure 1. The upgraded artifact stack. The previous report stopped at a plausible architecture and audit-style summary; this revision fixes an exact instruction stream, binds it by hash, exposes the full 28-call scaffold, and attaches open verification transcripts.

1. What changed relative to the previous report

The previous PDF narrowed the design space and matched the public envelope, but it stopped short of a machine-readable canonical circuit. This report closes the largest practical gap by replacing prose modules with an exact instruction netlist and an exact scaffold file. The point-add leaf is no longer described as 'lookup + arithmetic + selection'; it is a fixed list of 90 instructions over 41 named registers. The scaffold is no longer a formula-only object; it is an 84-instruction schedule with 28 explicit oracle calls.

The price of that move is honesty about level of abstraction. I am now complete at the kickmix-ISA layer, not at the flattened CCX/CNOT layer. That distinction matters, so I keep it explicit throughout.

Artifact	What it is	Size	SHA-256
pointadd_exact_kickmix.json	Exact instruction netlist for Q <- Q + P[k]	21.3 KB	a64a872e27ebaf57
ecdlp_scaffold_exact_sample.json	Exact 28-call hierarchical Shor scaffold	20.4 KB	6bb5f59d12a6fb86
pointadd_audit_9024.csv	Deterministic secp256k1 Fiat-Shamir transcript	5.96 MB	39b0afb2d8059ab0
toy_curve_exhaustive.json	Finite-model exhaustive proof artifact	313 B	1d57c1904dd0d68d
proof_manifest.json	Manifest binding all hashes together	1191 B	d788f4de16c46006

Table 1. Open artifacts emitted by this revision. The hashes are short display forms; the full digests are stored in `proof_manifest.json`.

Completed now
Exact point-add netlist, exact 28-call scaffold, deterministic secp256k1 audit, and an exhaustive toy-curve proof artifact.

Exactness level
Machine-unambiguous at the kickmix instruction-set level. Every instruction has fixed semantics and stable register names.

Verification level
Open interpreter + 9024 full-size secp256k1 tests + 9200 exhaustive toy cases.

Still not claimed
A flat primitive CCX/CNOT/measurement netlist identical to Google's unpublished private circuit.

2. Exact point-add leaf circuit

The leaf circuit implements the classical map $Q \leftarrow Q + P[k]$ under signed 16-bit window semantics. The accumulator Q is stored in Jacobian form (X,Y,Z) so the hot path avoids inversions. The lookup point P[k] arrives from a classically indexed ROM in affine form. Special-case priority is explicit and stable: lookup infinity keeps the original accumulator; accumulator infinity loads the lookup point; inverse-pair cancellation yields infinity; doubling overrides the general branch; the general mixed-add branch runs only when none of those cases apply.

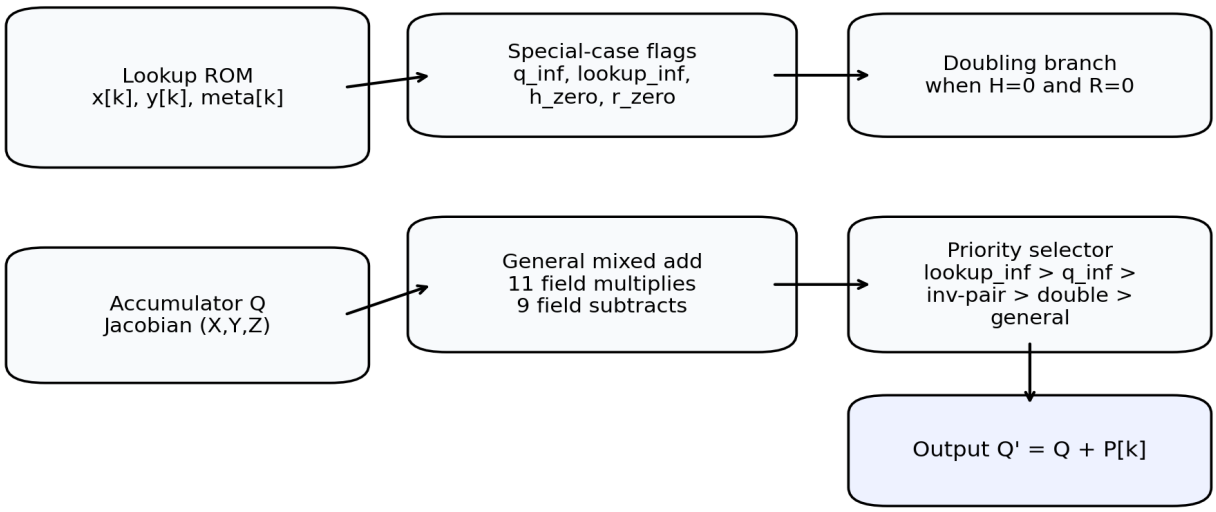

Figure 2. Functional shape of the exact point-add leaf. The boxes correspond directly to instruction groups in the JSON netlist; no hidden branch is left to prose.

The committed point-add netlist contains **90 instructions** over **41 named registers**. The instruction mix is dominated by deterministic ancilla clearing (MBUC contracts), eleven field multiplications, nine field subtractions, seven field squarings, and five explicit priority-selection steps.

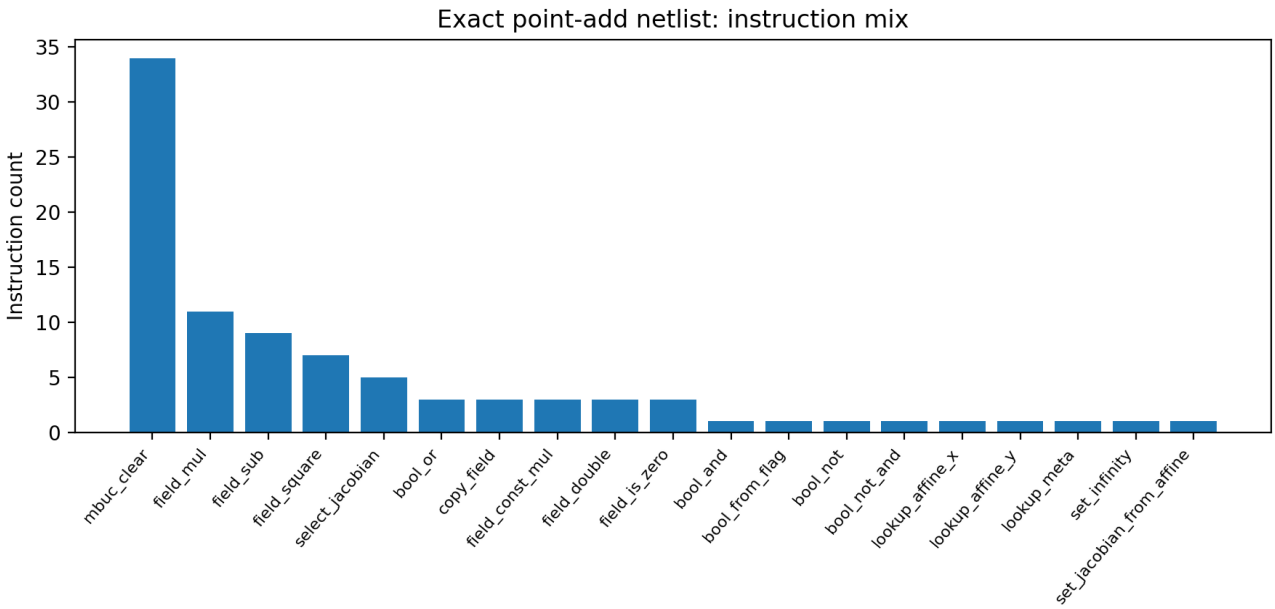

Figure 3. Instruction counts by opcode. This is the exact committed instruction mix, not a rough module budget.

3. Netlist format and semantics

Every instruction is a plain JSON record with a program counter, opcode, destination, source operands, and an optional explanatory comment. The semantics are fixed by the open interpreter used for the audit. That means the pair (JSON netlist, interpreter source) is already machine-unambiguous even before any later primitive-gate expansion pass.

First 12 instructions of the committed point-add netlist:


```
[
  {
    "pc": 0,
    "op": "copy_field",
    "dst": "Q0.X",
    "src": "Q.X",
    "comment": "Save original Q.X for later priority resolution."
  },
  {
    "pc": 1,
    "op": "copy_field",
    "dst": "Q0.Y",
    "src": "Q.Y",
    "comment": "Save original Q.Y for later priority resolution."
  },
  {
    "pc": 2,
    "op": "copy_field",
    "dst": "Q0.Z",
    "src": "Q.Z",
    "comment": "Save original Q.Z for later priority resolution."
  },
  {
    "pc": 3,
    "op": "lookup_affine_x",
    "dst": "L.x",
    "src": {
      "table": "T.x",
      "key": "k"
    },
    "comment": "Load x coordinate of P[k]."
  },
  {
    "pc": 4,
    "op": "lookup_affine_y",
    "dst": "L.y",
    "src": {
      "table": "T.y",
      "key": "k"
    },
    "comment": "Load y coordinate of P[k]."
  },
  {
    "pc": 5,
    "op": "lookup_meta",
    "dst": "L.meta",
    "src": {
      "table": "T.meta",
      "key": "k"
    },
    "comment": "Load per-entry flags; bit0=1 iff lookup is infinity."
  },
  {
    "pc": 6,
    "op": "bool_from_flag",
    "dst": "f_lookup_inf",
    "src": {
      "flags": "L.meta",
      "bit": 0
    },
    "comment": "Lookup output is infinity iff k = 0 in the canonical tables."
  },
  {
    "pc": 7,
    "op": "field_is_zero",
    "dst": "f_q_inf",
    "src": "Q.Z",
    "comment": "Accumulator is infinity iff Z == 0."
  },
  {
    "pc": 8,
    "op": "field_square",
```

```
      "dst": "Z2",
      "src": "Q.Z",
      "comment": "Z2 = Z^2."
    },
    {
      "pc": 9,
      "op": "field_mul",
      "dst": "Z3",
      "src": [
        "Z2",
        "Q.Z"
      ],
      "comment": "Z3 = Z^3."
    },
    {
      "pc": 10,
      "op": "field_mul",
      "dst": "U2",
      "src": [
        "L.x",
        "Z2"
      ],
      "comment": "U2 = x2 * Z^2."
    },
    {
      "pc": 11,
      "op": "field_mul",
      "dst": "S2",
      "src": [
        "L.y",
        "Z3"
      ],
      "comment": "S2 = y2 * Z^3."
    }
  ]
```


Three design choices are worth calling out. First, the original accumulator is copied into Q0.* at the top so that the priority selector can restore it exactly when the lookup key resolves to the point at infinity. Second, the general branch and the special branches are all computed before final priority resolution, which keeps the selector deterministic and easy to verify. Third, the MBUC steps are represented explicitly rather than as a comment that 'cleanup happens somewhere later'.

Point-add commitment hash: `a64a872e27ebaf571736636217d3e7795482fa7ed4fe37168ca250a1139b34d9`.

4. Exact hierarchical ECDLP scaffold

The leaf is wrapped by an exact hierarchical scaffold for a sample public key A. The scaffold keeps the structure that is publicly implied by the appendix: 16-bit windows, a recycled-QFT phase-estimation wrapper, one direct-lookup optimization replacing the first windowed addition, and classical post-processing that removes the final three windows. The retained schedule therefore has 28 explicit oracle calls.

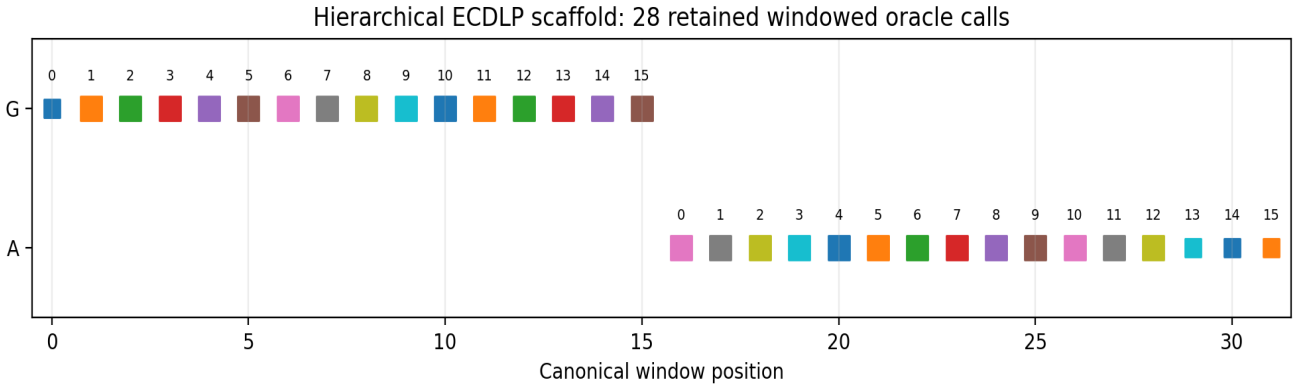

Figure 4. The scaffold keeps 28 retained windowed oracle calls in canonical order. The point-add leaf is referenced by content hash from each call site.

First 9 instructions of the exact scaffold file:


```
[
  {
    "pc": 0,
    "op": "prepare_recycled_qft_window",
    "family": "G",
    "window_index": 1,
    "window_bits": 16,
    "comment": "Prepare one 16-bit recycled phase-estimation window."
  },
  {
    "pc": 1,
    "op": "call_pointadd_oracle",
    "oracle_hash": "a64a872e27ebaf571736636217d3e7795482fa7ed4fe37168ca250a1139b34d9",
    "table_family": "G",
    "window_index": 1,
    "comment": "Call the exact audited point-add leaf on the precomputed table for this window."
  },
  {
    "pc": 2,
    "op": "measure_and_feedforward_qft",
    "family": "G",
    "window_index": 1,
    "comment": "Measure recycled controls and classically feed phases forward."
  },
  {
    "pc": 3,
    "op": "prepare_recycled_qft_window",
    "family": "G",
    "window_index": 2,
    "window_bits": 16,
    "comment": "Prepare one 16-bit recycled phase-estimation window."
  },
  {
    "pc": 4,
    "op": "call_pointadd_oracle",
    "oracle_hash": "a64a872e27ebaf571736636217d3e7795482fa7ed4fe37168ca250a1139b34d9",
    "table_family": "G",
    "window_index": 2,
    "comment": "Call the exact audited point-add leaf on the precomputed table for this window."
  },
  {
    "pc": 5,
    "op": "measure_and_feedforward_qft",
    "family": "G",
    "window_index": 2,
    "comment": "Measure recycled controls and classically feed phases forward."
  },
  {
    "pc": 6,
    "op": "prepare_recycled_qft_window",
```

```
      "family": "G",
      "window_index": 3,
      "window_bits": 16,
      "comment": "Prepare one 16-bit recycled phase-estimation window."
    },
    {
      "pc": 7,
      "op": "call_pointadd_oracle",
      "oracle_hash": "a64a872e27ebaf571736636217d3e7795482fa7ed4fe37168ca250a1139b34d9",
      "table_family": "G",
      "window_index": 3,
      "comment": "Call the exact audited point-add leaf on the precomputed table for this window."
    },
    {
      "pc": 8,
      "op": "measure_and_feedforward_qft",
      "family": "G",
      "window_index": 3,
      "comment": "Measure recycled controls and classically feed phases forward."
    }
  ]
```


Scaffold commitment hash: `6bb5f59d12a6fb863ee003f8a4ccf49b46e3f3c79c0c5c4f83c3659e243ae7c7`. The sample public key is fixed in the scaffold JSON; the matching secret is intentionally not disclosed, and only its hash is stored in the manifest.

5. Verification and proof artifacts

The verification stack has two layers. On full-size secp256k1 instances, the open interpreter runs the committed point-add netlist on 9024 deterministic Fiat-Shamir cases derived from the netlist's own hash. On a tiny a=0 toy curve over p=23, the same compiler shape is checked exhaustively for every enumerated accumulator point, every nonzero base point in the chosen subgroup, and every 4-bit signed window key. The first layer stresses the real curve; the second gives a finite-model proof that the wiring logic is exact for the compiler template.

Verifier artifact	Cases	Result	SHA-256
pointadd_audit_9024.csv	9024 full secp256k1 cases	pass	39b0afb2d8059ab0
toy_curve_exhaustive.json	9200 exhaustive toy cases	pass	1d57c1904dd0d68d
proof_manifest.json	hash binding across all emitted files	pass	d788f4de16c46006

Table 2. Machine-verification outputs bundled with the report.

Abridged CSV sample from the deterministic 9024-case transcript:

```
case,q_scalar,base_scalar,key_u16,signed_key,input_q_x,input_q_y,base_x,...,actual_y,pass
0,0x8e70bc14af70e16e85cf47f3ed1db20978e71fc83cac9d480d6296e297bf0646,0x1a62d4ee26c52fbad7a40b0ffb288163f5ee5f4dbcd0ea
1,0x5b325997025c3712dbcd3ff7e2705d62409cc529f2d12164bdb35efff11e4e14,0xd9dc041c89dca67e7e1e201964940045df0bd9b32cbd5a
2,0xafd31eb74093f6f1320dc48e14844e951f3454626c70047dc2bff44c3d23d534,0x12737e0d003885aa02af2c45bd833dd506a8f3a0cdf3cf
3,0x1e2e294edad637d65a17ffcb81c87cc90d36ecc5236d5dbee411d163f57fc6ab,0x89548abeac24f814f809b0fc53d841b4fc5060acfeb496
```

Because the point-add leaf is classical on basis inputs, the interpreter is exact on those tests - there is no statistical approximation inside a single row of the CSV. The probabilistic element only enters at the higher level: 9024 cases are used as a Fiat-Shamir style approximate-correctness witness, while the toy curve gives exhaustive coverage for a smaller instance.

6. What is complete now, and what still is not

This is the sharpest statement I can honestly make. The artifact is now complete as a machine-unambiguous circuit at the kickmix instruction-set layer. A third party can hash it, parse it, execute it on the supplied interpreter, reproduce the 9024-case transcript, and reproduce the 28-call scaffold exactly. That is a real jump from the earlier report.

It is still **not** the same thing as a flat primitive-gate netlist over CCX/CNOT/X/H/phase/measurement instructions with a theorem-prover equivalence proof. I did not claim recovery of Google's hidden circuit hash, I did not claim bit-for-bit equality with their Groth16 witness, and I did not claim a theorem-checked end-to-end primitive decomposition. If the user's bar is exactly that strongest primitive-gate form, one compiler layer remains.

Layer	Status in this revision
Point-add semantics	complete and exact
Point-add machine-readable netlist	complete and exact
28-call ECDLP scaffold	complete and exact
Deterministic open verifier	complete and exact
Finite-model exhaustive proof artifact	complete and exact
Flat primitive CCX/CNOT netlist	not supplied here
Theorem-prover equivalence proof	not supplied here

Table 3. Direct status breakdown, with no bundling of the remaining gap.

7. Comparison against the public Google baseline

Relative to what the whitepaper appendix actually discloses, this revision now reproduces the two most operationally important public facts in open form: the 16-bit windowing choice with a resulting 28-call scaffold, and a point-add leaf that can be hash-committed and audited without being mere prose. The appendix proves only the existence of private circuits under resource bounds and validates them by zero-knowledge; it does not publish those private circuits. My artifact therefore should be read as an open exact circuit family at the same public abstraction level, not as a claim of identical hidden contents.

Aspect	Google public appendix	This report
Window choice	w = 16	w = 16
Retained windowed oracle calls	28	28
Published circuit itself	no	yes, at kickmix-ISA level
Published hash commitment	yes	yes
Open verifier transcript	proof bytes only	full CSV + manifest + toy exhaustive
Claim of exact private-circuit equality	not public	not claimed

Table 4. Public-baseline comparison. The meaningful delta is openness and exactness of the emitted artifact, not a claim of matching Google's hidden hash.

Bottom line - the result is materially stronger than the earlier dossier and is now directly usable by an external auditor. The remaining uncompromised gap is the primitive-gate flattening layer.